

单链表强化

//1、尽可能高效地删除递增链表中重复的结点，仅保留第一个

```
void Delete(Linklist &L){
    if(L==NULL||L->next==NULL){
        return;
    }
    LNode*p=L->next;
    LNode*q=p->next;
    while(p!=NULL){
        while(q!=NULL&&q->data==p->data){
            LNode*temp = q;
            q=q->next;
            free(temp);
        }
        p->next=q;
        p=p->next;
        if(q!=NULL) q=q->next;
    }
}
```

//2、删除单链表中重复的结点，仅保留第一个

//暴力破解法

```
void Delete(Linklist&L){
    if(L==NULL||L->next==NULL){
        return ;
    }
    LNode*p=L->next;
    while(p!=NULL){
        LNode*q=p;
        while(q->next!=NULL){
            if(q->next->data==p->data){
                LNode*temp = q->next;
                q->next=temp->next;
                free(temp);
            }else{
                q=q->next;//只有没删时才移动
            }
        }
        p=p->next;
    }
}
```

```

    }
}
//哈希表解法
void Delete(Linklist&L){
    if(L==NULL||L->next==NULL){
        return ;
    }
    bool seen[MAXSIZE] = {false}; //记录已经出现过的值
    LNode*p=L; //指向前驱结点
    while(p->next!=NULL){
        int val = p->next->data;
        if(seen[val]){
            LNode*temp = p->next; //暂存待删除结点
            p->next = temp -> next;
            free(temp); //释放结点
        }else{
            seen[val] = true; //第一次访问，则记录值
            p=p->next;
        }
    }
}

```

//用单链表保存 m 个整数，且 $|data| < n$ ，设计时间上尽可能高效的算法，对于 $data$ 绝对值相等的点，仅保留第一次出现的点

```

void Delete(Linklist&L,int n){
    if(L==NULL||L->next==NULL){
        return ;
    }
    bool seen[n] = {false}; //记录已经出现过的值
    LNode*p=L; //指向前驱结点
    while(p->next!=NULL){
        int val = abs(p->next->data);
        if(seen[val]){
            LNode*temp = p->next; //暂存待删除结点
            p->next = temp -> next;
            free(temp); //释放结点
        }else{
            seen[val] = true; //第一次访问，则记录值
            p=p->next;
        }
    }
}

```

//将一个带头结点的单链表 A 分解成两个带头结点的单链表 A 和 B，使 A 中含奇数位置元素，B 中含偶数位置的元素，且相对位置不变

```

void Divide(Linklist&A,Linklist&B){
    if(A==NULL||A->next==NULL){
        return ;
    }
    B = (Linklist)malloc(sizeof(LNode));    // 给 B 分配头结点
    B->next = NULL;                          //初始化
    int pos = 1;
    LNode*r = A;
    LNode*p = A->next;
    LNode*q = B;
    r->next = NULL;
    while(p!=NULL){
        if(pos%2!=0){
            r->next = p;
            r = p;
            p = p->next;
            r->next = NULL;                //断开尾指针
        }else{
            LNode*temp = p;
            p = p->next;
            temp ->next = NULL;            //把 temp 断开
            q->next = temp;
            q = q->next;
        }
        pos++;
    }
    //最后统一断开尾指针
    r->next = NULL;
    q->next = NULL;
}

```

//将一个单链表(a1,b1,a2,b2,....,an,bn)拆分成(a1,a2,a3,....,an)和(bn,bn-1,....,b2,b1)

```

void Divide(Linklist&A,Linklist &B){
    if(A==NULL||A->next==NULL) return ;
    B = (Linklist)malloc(sizeof(LNode));
    B->next = NULL;
    int pos = 1;
    LNode*r = A;
    LNode*p = A->next;
    r->next = NULL;
    LNode*q = B;
    while(p!=NULL){
        if(pos%2!=0){

```

```

        r->next = p;
        r = p;                                //先连接
        p = p->next ;
        r->next = NULL;
    }else{
        LNode*temp = p;
        p = p->next ;
        temp->next = q->next;
        q->next = temp;
    }
    pos++;
}
r->next = NULL;
}

```

//两个递增有序的单链表，设计算法将 B 合并到 A 中，仍保持一个非递减有序的链表

```

Linklist&Merge(Linklist&A,Linklist &B){

```

```

    LNode*p = A->next;
    LNode*q = B->next;
    LNode*r = A;
    while(p!=NULL&&q!=NULL){
        if(p->data<q->data){
            r->next =p;
            p = p->next;
        }else{
            r->next = q;
            q = q->next;
        }
        r = r->next;
    }
    if(p!=NULL) r->next = p;
    else r->next = q;

```

```

    free(B);
    return A;
}

```

两个递增有序的单链表，设计算法将 B 合并到 A 中，仍保持一个非递增有序的链表

```

Linklist &Merge(Linklist&A,Linklist&B){

```

```

    LNode*p = A->next;
    LNode*q = B->next;
    LNode*r = A;
    r->next = NULL;

```

```

while(p!=NULL&&q!=NULL){
    LNode*temp = NULL;
    if(p->data<=q->data){
        temp = p;
        p=p->next;
    }else{
        temp = q;
        q = q->next;
    }
    temp->next = r->next;
    r->next = temp;
}
while(p!=NULL){
    LNode*temp = p;
    p=p->next;
    temp->next = r->next;
    r->next = temp;
}
while(q!=NULL){
    LNode*temp = q;
    q=q->next;
    temp->next = r->next;
    r->next = temp;
}
free(B);
return A;
}

```

//简化版本

```

Linklist &Merge(Linklist &A, Linklist &B) {

```

```

    LNode *p = A->next;
    LNode *q = B->next;
    LNode *r = A;
    r->next = NULL;

```

// 合并 + 头插

```

while(p != NULL || q != NULL){
    LNode *temp = NULL;
    if(q == NULL || (p != NULL && p->data <= q->data)){
        temp = p;
        p = p->next;
    } else {
        temp = q;
        q = q->next;
    }
}

```

```

        temp->next = r->next;
        r->next = temp;
    }

    free(B); // 释放 B 的头结点
    return A;
}

//A, B 两个链表递增有序, 从 A,B 中找出公共元素产生单链表 C, 要求: 不破坏 A 和 B 的
//结点
Linklist CommonList(Linklist&A,Linklist&B,Linklist&C){
    LNode*p=A;
    LNode*q=B;
    LNode*r=C;
    while(p->next!=NULL&&q->next!=NULL){
        if(p->next->data<q->next->data){
            p=p->next;
        }else if(p->next->data>q->next->dara){
            q=q->next;
        }else{
            LNode*temp = (LNode*)malloc(sizeof(LNode*));
            temp->data = p->data;
            temp->next = NULL;
            r->next = temp;
            r=temp;
            p=p->next;
            q=q->next;
        }
    }
    return C;
}

```

//A,B 两个单链表递增有序, 从 A, B 中找出公共元素存放在 A 链表中。要利用 A 链表的原有结点空间

```

Linklist CommonList(Linklist &A, Linklist &B) {
    LNode* p = A->next;
    LNode* q = B->next;
    LNode* r = A;
    r->next = NULL;

    while (p != NULL && q != NULL) {
        if (p->data < q->data) {
            LNode* tmp = p;
            p = p->next;

```

```

        free(tmp); // 释放 A 中没用的节点
    } else if (p->data > q->data) {
        q = q->next;
    } else {
        LNode* temp = p;
        p = p->next;
        q = q->next;
        temp->next = NULL;
        r->next = temp;
        r = temp;
    }
}

// 把 A 剩余没用的节点释放掉
while (p != NULL) {
    LNode* tmp = p;
    p = p->next;
    free(tmp);
}

r->next = NULL;
return A;
}

//A、B 两个单链表递增有序，从 A、B 中找出 A 有但 B 没有的元素并存放于 A 链表中，且
要求利用 A 的原有空间
Linklist A_sub_B(Linklist&A,Linklist&B){
    LNode*p=A;
    LNode*q=B->next;
    while(p->next!=NULL&&q!=NULL){
        if(p->next->data<q->data){
            p=p->next;
        }else if(p->next->data>q->data){
            q=q->next;
        }else{
            LNode*temp=p->next;
            p->next=temp->next;
            free(temp);
            q=q->next;
        }
    }
}
return A;
}

```

//A、B 两个单链表递增有序，从 A、B 中找出所有元素后再去重并存放于 A 链表中，且要求利用 A 和 B 的原有空间

```
Linklist Merge(Linklist&A,Linklist&B){
    LNode*p=A;
    LNode*q=B;
    //拼接
    while(p->next!=NULL||q->next!=NULL){
        if(p->next==NULL||p->next!=NULL&&p->next->data<=q->next->data){
            LNode*temp = q->next;
            q=temp->next;
            temp->next=p->next;
            p->next=temp;
            p=temp;
        }
        else{
            p=p->next;
            q=q->next;
        }
    }
    //去重
    p=A->next;
    q=p->next;
    while(q!=NULL){
        while(q->data==p->data){
            LNode*temp=q;
            q=q->next;
            free(temp);
        }
        p->next=q;
    }
    return A;
}
```

//假定采用带头结点的单链表保存单词，设 str1 和 str2 分别指向两个单词所在链表的头结点，请设计一个时间上尽可能高效的算法，找出这两个链表的共同后缀的起始位置

```
LNode FindCommontail(Linklist&A,Linklist&B){
    LNode*str1=A;
    LNode*str2=B;
    int len1=0,len2=0;
    while(str1!=NULL){
        str1=str1->next;
```



```

        len1++;
    }
    while(str2!=NULL){
        str2=str2->next;
        len2++;
    }
    str1=A->next;
    str2=B->next;
    if(len1>len2){
        for(int i=0;i<len1-len2;i++){
            str1=str1->next;
        }
    }else if(len1<len2){
        for(int i=0;i<len2-len1;i++){
            str2=str2->next;
        }
    }
    LNode*CommonStart=NULL;
    while(str1!=NULL&&str2!=NULL){
        if(str1==str2){
            CommonStart=str1;
            return CommonStart;
        }
        str1=str1->next;
        str2=str2->next;
    }
    return NULL;
}

```

//查找链表倒数第 k 个结点，若查找成功，则输出该结点的 data，并返回 1，否则返回 0

```

int Fink_k(Linklist A,int k{
    LNode*p=A->next;
    int len=0;
    while(p!=NULL){
        p=p->next;
        len++;
    }
    if(k>len||k<=0){return 0;}
    for(int i=0;i<len-k;i++){
        p=p->next;
    }
    printf("%d",p->data);
    return 1;
}

```

```

} //O(n2)
//方法二
int Find_k(Linklist A, int k) {
    LNode *fast = A->next, *slow = A->next;
    int i = 0;
    while (i < k && fast != NULL) {
        fast = fast->next;
        i++;
    }
    if (i < k) return 0; // 不足 k 个结点

    while (fast != NULL) {
        fast = fast->next;
        slow = slow->next;
    }
    printf("%d", slow->data);
    return 1;
} //O(n)

```

//两个递增有序的循环单链表，设计算法将 B 合并到 A 中，仍然保证一个非递减的循环单链表

```

Linklist Merge(Linklist&A, Linklist&B){
    LNode*p=A->next;
    while(p->next!=A){
        p=p->next;
    }
    p->next = NULL;
    LNode*q=B->next;
    while(q->next!=B){
        q=q->next;
    }
    q->next = NULL;
    p=A;
    q=B;
    LNode*r=A;
    while(p->next!=NULL&&q->next!=NULL){
        if(p->next->data<=q->next->data){
            r->next=p->next;
            p->next=p->next->next;
        }else{
            r->next=q->next;
            q->next=q->next->next;
        }
        r=r->next;
    }
}

```

```
}  
r->next=(p->next!=NULL?p->next:q->next);  
  
while(r->next!=NULL){  
    r=r->next;  
}  
r->next =A;  
free(B);  
return A;  
}
```